Lab 3 - Motion Control of the Parrot AR.Drone (2025/26)

Course notes – 2025/26 labs

Contents

Source files:

- `25_26_UAVs_Lab3.pdf` (5 pages) - lab handout, "Unmanned Aerial Vehicles, M.Sc. in Aerospace Engineering, 2025/2026 - First Semester, Motion Control of the Parrot AR.Drone, Laboratory Guide, November 2025", IST Lisboa.
- `DevKit_Lab3/ARDroneSimulinkDevKit_V1.1/guide.pdf` (3 pages) - generic devkit user guide.
- `DevKit_Lab3/ARDroneSimulinkDevKit_V1.1/start_here_CTRL.m`, `lib/euler2R.m`, `lib/getWaypoints.m`, `lib/setupARModel.m`, `simulation/setupBaseModel.m`, `simulation/setupHoverSim.m`, `simulation/setupTTSim.m`, `simulation/setupWPTrackingSim.m`, `wifiControl/setupHover.m`, `wifiControl/setupTT.m`, `wifiControl/setupWPTracking.m`.
- `.slx` Simulink models (binary, not parsed): `lib/ARBlocks.slx`, `simulation/ARDroneBaseSim.slx`, `simulation/ARDroneHoverSim.slx`, `simulation/ARDroneTTSim.slx`, `simulation/ARDroneWPTrackingSim.slx`, `wifiControl/ARDroneHover.slx`, `wifiControl/ARDroneTT.slx`, `wifiControl/ARDroneWPTracking.slx`.

1 Lab handout - objectives and tasks

Objectives (Sec 1.1): (1) design a trajectory-tracking controller for a generic quadrotor; (2) add an adaptation law to reject an unknown (constant) disturbance;

(3) implement a modified version of that controller on the AR.Drone DevKit; (4) implement a simple trajectory planner generating straight-line and circular reference trajectories.

Logistics (Sec 1.2): questions are tagged (T) theoretical (solve before the lab session) or (L) laboratory (complete during sessions, using the provided MATLAB/Simulink package). Deliverable: a single-column PDF report, **max 15 pages**, plus the MATLAB code, submitted via Fenix; must use the official cover page (`25_26_UAVs_Lab_report_cover_page.docx`, in this same `labs-25-26/` folder). Sec 1.3 is a standard academic-integrity clause (report must be original work).

1.1 Section 2 - Trajectory Tracking Control (all theoretical, T)

This section is a **complete, self-contained re-derivation** of the LQR-based trajectory-tracking controller from the lectures, extended with an adaptive disturbance-rejection term. It reuses the exact quadrotor linear-motion equation from the course:

- **2.1**: Start from $\ddot{p} = ge_3 - R(T/m)e_3$ with $R = R_z(\psi)R_y(\theta)R_x(\phi)$ (identical convention to `../../lectures-22-23/notes/01-foundations.pdf`). Change of variables to $\ddot{p} = ge_3 - R_z(\psi)u^*$ (rotate the virtual input into the yaw-aligned frame), then invert to get T, θ_r, ϕ_r as functions of u^*. This is the same "extract thrust magnitude + attitude reference from a desired acceleration" trick as the differential-flatness result in Ch2, but expressed slightly differently: here only the yaw rotation $R_z(\psi)$ is factored out (since it's directly measured/controlled), leaving u^* to be produced by an *inner* attitude loop assumed to already track (ϕ_r, θ_r) on a fast timescale - i.e. the same two-timescale (5-10x bandwidth separation) cascade assumption as `../../lectures-22-23/notes/02-control.pdf` (Ch3).
- **2.2-2.3**: Standard LQR design on the double-integrator error system $\tilde{x} = [\tilde{p}; \tilde{v}]$, $\dot{\tilde{x}} = A\tilde{x} + Bu$. Students must derive $K = (1/\rho)B'P$ from the Riccati equation $A'P + PA - (1/\rho)PBB'P + Q = 0$ and prove closed-loop stability via $V_1 = \tilde{x}'P\tilde{x}$, $\dot{V}_1 = -\tilde{x}'Q^*\tilde{x}$ - this is a direct, numbered application of the **LQR theorem** from `02-control.pdf` (Ch4), just with cost weight ρ on the control instead of a general R.
- **2.4-2.6**: Extends the model to $\ddot{p} = ge_3 - R_z(\psi)u^* + w$ with unknown constant disturbance w (e.g. wind), and an adaptive law $u^* = R_z(-\psi)(K\tilde{x} + ge_3 + \hat{w} - \ddot{p}_d)$. Students must propose an adaptation law for $\dot{\hat{w}}$ using the augmented Lyapunov function $V_2 = V_1 + (1/k_w)\|\tilde{w}\|^2$ and prove **global asymptotic stability** of the augmented state $\xi = [\tilde{x}; \tilde{w}]$. This is a textbook instance of the **adaptive-control pattern** from `../../lectures-22-23/notes/04-guidance-nonlinear.pdf` (Ch7 Part II, "Lyapunov analysis and adaptive control": $V_2 = V(x) + (1/k_\theta)\tilde{\theta}^2$, adaptation law $\dot{\hat{\theta}} = k_\theta(\partial V/\partial x)g(x,u)$) - here the unknown parameter is a vector disturbance rather than a scalar model parameter, but the construction is identical: augment V, cancel the cross term, get $\dot{V}_2 \leq 0$ (not < 0, since LaSalle is needed - matching the note in Ch7 that adaptive parameter estimates are generally only bounded, not exactly convergent, unless "persistently exciting").
- **2.7**: Asks students to redraw the closed loop as the block diagram in Fig. 1 ($p_d \to [+/-] \to \bar{K}(s) \to [+\ \ddot{p}_d + w] \to 1/s^2 \to p$) and discuss the connection to

a **PID controller** - directly parallel to the Ch3 discussion (`02-control.pdf`) of how adding an integrator/adaptive state to a PD position loop produces PID-like disturbance rejection, and to the Ch7 backstepping result that backstepping "reproduces a PD-like tracking law but derived constructively."

- **2.8**: A short, separate exercise: design a feedback law for the (decoupled) yaw first-order dynamics to track a constant reference with **zero steady-state error** - i.e. requires integral action (internal model principle), same idea as the wind-rejection PID discussion in Ch3.

1.2 Section 3 - Implementation and simulation using the AR.Drone DevKit (all laboratory, L)

- An **updated DevKit** is provided for this lab (see Devkit section below): new `start_here_CTRL.m` entry point, new option **(4) Trajectory Tracking**, opening `ARDroneTTSim.slx` (simulation) or `ARDroneTT.slx` (Wi-Fi control). Two new Simulink blocks, `desired_trajectory` and `tt_controller` (the latter inside `Outer-loop Controller`), have their I/O interfaces defined but their **content is intentionally left (partially) empty** - implementing them (as MATLAB Function blocks) is the actual lab exercise. Neither `.m` script nor the `.slx` files contain numeric controller gains; all gains (K, k_w, k_w for the vertical proportional term, LOS parameters Δ,V) are left for the students to design and tune.
- **3.1**: The real DevKit inner loop only accepts $(\phi_r, \theta_r, w_r, \dot{\psi}_r)$ (roll/pitch angle references, vertical-velocity reference, yaw-rate reference) rather than a raw torque n - so the theoretical controller of Sec. 2 must be adapted: keep the (ϕ_r, θ_r) and yaw-rate expressions, but replace the vertical channel with a simple proportional law $w_r = -k_w(z - z_d)$ (justified because trajectories are assumed horizontal-plane with small roll/pitch).
- **3.2-3.3**: Implement `desired_trajectory` for a **horizontal straight-line** path at constant height, speed 0.1 m/s, constant yaw, starting from the drone's actual position when tracking is enabled (avoids a large initial position-error transient/actuation spike). Must also supply $\dot{p}_d$, $\ddot{p}_d$ (used by the LQR/adaptive law). Then test and tune gains.
- **3.4-3.6**: Switch to a **circular trajectory** (radius 1 m, angular rate 0.1 rad/s, constant yaw); then experiment with making yaw always point toward the circle's center, and discuss whether that alone achieves zero tracking error (it generally won't, since the position-tracking error dynamics don't depend on yaw - consistent with 2.8's remark that "yaw is not constrained by the linear dynamics"), motivating further controller changes if not.
- **3.7**: Replace the "reset desired position to current position" trick with a **line-of-sight (LOS) straight-line path-following** scheme: $\psi_d = \text{atan}(-(y - y_d)/\Delta)$, $\dot{x}_d = V\cos(\psi_d)$, $\dot{y}_d = V\sin(\psi_d)$. This is the **same LOS guidance concept** flagged (but not derived) as an alternative to the vector-field path-following law in `04-guidance-nonlinear.pdf` (Ch8) - here it appears as a fully worked, lab-specific instance, with Δ (lookahead-like distance) and V (nominal speed) as the tunable parameters whose effect on path convergence students must discuss - directly analogous to how $k_{path} \approx 1/R_{min}$ tunes the vector-field law's transition sharpness in Ch8.

2 Devkit guide - CTRL/TT-specific usage

The `guide.pdf` shipped with `DevKit_Lab3` is byte-identical to the one shipped with `DevKit_Lab1` (confirmed via `md5sum`: both `83c7109e5b163664c8b2eeafe100f558`). It is the original, unmodified "ARDrone Simulink Development Kit" guide by D. Escobar Sanabria & P. J. Mosterman (University of Minnesota / MathWorks, v1.0, Sept. 2013), and it documents only the original three examples: Waypoint Tracking, Hovering/Position Control, and Baseline Simulation. **It does not mention `start_here_CTRL.m`, the Trajectory Tracking option, `ARDroneTTSim.slx/ARDroneTT.slx`, or the `desired_trajectory/tt_controller` blocks at all** - the Lab3 handout's own Section 3 is the only documentation for these additions; the generic `guide.pdf` was not updated for this lab. Practical implication: any student (or future session) looking for usage instructions for the trajectory-tracking mode must rely on the lab handout, not the devkit guide, and on reading `start_here_CTRL.m` directly.

The guide's generic content that *does* still apply to Lab3: requirements (MATLAB/Simulink R2013b 32/64-bit, Real-Time Windows Target R2013b via `rtwintgt -setup`, Parrot AR.Drone 2.0), the `ARBlocks.slx` library description (`ARDrone Simulation Block` = system-identified vehicle-dynamics model; `ARDrone Wi-Fi Block` = Real-Time-Windows-Target-based Wi-Fi interface, same I/O as the simulation block for easy sim-to-real transfer), the vehicle/frame diagram (body frame X_b along the heading direction; inertial frame X_e,Y_e), and the video-streaming `ffmpeg/ffplay` commands (`http://192.168.1.1:5555`, unrelated to control but part of the same devkit).

3 Devkit scripts - control-specific additions vs Lab1's devkit

Script	What it configures	Notable content
`start_here_CTRL.m`	Entry point (replaces Lab1's plain `start_here.m`)	Adds **option (4) Trajectory Tracking** under both Simulation and Wi-Fi-control menus, dispatching to `setupTTSim`/`setupTT`. Otherwise structurally identical to Lab1's menu (options 1-3 unchanged: baseline sim, hover, waypoint tracking). No numeric gains.
`lib/euler2R.m`	Euler angles → rotation matrix (**new in Lab3's devkit** - absent from Lab1/Lab2)	See "Connections to lecture material" below - implements $R = R_z(\psi)R_y(\theta)R_x(\phi)$.
`lib/getWaypoints.m`	Waypoint list $(X_e, Y_e, h, \psi_d$, waiting time) for the WP-tracking example	Unchanged from Lab1: 11 waypoints hard-coded (mostly hover at $(0, 0, 1)$ with one excursion to $(\pm 1.5, 0, 1)$ and one to height 1.8 m). Not used by the TT example (its setup script comments out the `getWaypoints()` call).

Script	What it configures	Notable content
`lib/setupARModel.m`	Loads linear state-space/transfer-function models (`ssRoll`, `ssRoll2V`, `ssPitch`, `ssPitch2U`, `ssYaw`, `ssH`) from `.mat` files	Identical across Lab1/2/3 - this is the **system-identified inner-loop plant model** (roll/pitch/yaw/height dynamics) that the lecture's Ch3/Ch4 controllers are designed against; the actual numeric state-space matrices are inside the `.mat` files, not inspected here (binary).
`simulation/setupBaseModel.m`, `setupHover-Sim.m`, `setupWP-TrackingSim.m`	Baseline/hover/waypoint simulation setups	Unchanged in structure from Lab1 (same `FMS.Ts = 0.065` s baseline/WP, 0.03 s hover; `timeDelay = 4*Ts`; `simDT = 0.005`). No control gains present in the `.m` files - any gains live inside the `.slx` model blocks (not text-readable).
`simulation/setupTTSim.m` (**new**)	Trajectory-tracking simulation setup	Same skeleton as the others (`FMS.Ts=0.03`, `timeDelay=4*Ts`, `simDT=0.005`), loads `ARDroneTTSim.slx`. Notably **comments out** the `getWay-points()` call (trajectory tracking doesn't use the waypoint list - it uses the `desired_trajectory` block instead).
`wifiControl/setupHover.m`, `setupWPTrack-ing.m`	Wi-Fi hover / waypoint-tracking setup	Unchanged from Lab1 (`sampleTime=0.03` hover, `0.065` WP-tracking); both carry the same "position estimate is inaccurate - obtained by integrating an inaccurate velocity estimate from onboard optical flow" warning, directly relevant to why Lab2's estimation work matters for Lab3's control performance.
`wifiControl/setupTT.m` (**new**)	Wi-Fi trajectory-tracking setup	Minimal (`sampleTime=0.03`, loads `ARDroneTT`), same structure as `setupHover.m`/`setupWPTracking.m` but without the waypoint list (uses `desired_trajectory` block instead).

No PID/LQR/adaptation numeric gains appear anywhere in the `.m` files - by design, the controller (`tt_controller`) and trajectory generator (`desired_trajectory`) are **MATLAB Function blocks left empty inside the `.slx` files**, to be filled in by the student per the handout's Sec. 3 instructions. This matches the handout's explicit statement: "the content is partially empty. To implement and simulate the trajectory tracking controller, both blocks need to be filled in the form of Matlab Functions."

4 Simulink models present

- `lib/ARBlocks.slx` - shared library (ARDrone Simulation Block, ARDrone Wi-Fi Block), same across all three labs.
- `simulation/ARDroneBaseSim.slx`, `ARDroneHoverSim.slx`, `ARDroneWPTrack-ingSim.slx` - unchanged baseline/hover/waypoint-tracking simulation models (present identically in Lab1's devkit).

- `simulation/ARDroneTTSim.slx` (**new, Lab3-only**) - trajectory-tracking simulation model; contains the `desired_trajectory` block and the `Outer-loop Controller` subsystem (with `tt_controller` inside), both partially empty per the handout.
- `wifiControl/ARDroneHover.slx`, `ARDroneWPTracking.slx` - unchanged Wi-Fi-control hover/waypoint models.
- `wifiControl/ARDroneTT.slx` (**new, Lab3-only**) - Wi-Fi-control counterpart of `ARDroneTTSim.slx`, real-time deployable version.

(Contents not parsed - `.slx` files are Simulink's zipped-XML binary model format; the handout's text description above is the only available account of what's inside `desired_trajectory` and `tt_controller`.)

5 Connections to lecture material

- **`euler2R.m` matches the course's Euler-angle convention exactly.** Expanding $\lambda = [\phi, \theta, \psi]$, the script computes $R = R_z(\psi)R_y(\theta)R_x(\phi)$ (row/column-by-column verified: e.g. $R(1,1) = \cos(\theta)\cos(\psi)$, $R(3,1) = -\sin(\theta)$, $R(3,2) = \sin(\phi)\cos(\theta)$, $R(3,3) = \cos(\phi)\cos(\theta)$) - this is precisely the ZYX Euler rotation matrix documented in `../../lectures-22-23/notes/01-foundations.pdf` (Ch1): ${}^I_B R = R_z(\psi)R_y(\theta)R_x(\phi)$. No sign or ordering discrepancy - a lab report can cite the lecture notes' rotation-matrix formula directly when explaining/using this function.
- **Section 2's LQR trajectory-tracking design is a direct, numbered walk-through of the LQR theorem** in `../../lectures-22-23/notes/02-control.pdf` (Ch4: $K = R^{-1}B^T P$, $A^T P + PA + Q - PBR^{-1}B^T P = 0$, minimum cost $x'(0)Px(0)$) - here specialized to $R \to \rho$ (scalar cost weight) and the position-error double-integrator (A, B) pair.
- **Section 2.4-2.6's adaptive disturbance-rejection law is a direct application of the adaptive-control pattern** in `../../lectures-22-23/notes/04-guidance-nonlinear.pdf` (Ch7 Part II) - augmented Lyapunov function, gradient-type adaptation law, LaSalle-based GAS proof of the augmented $(\tilde{x}, \tilde{w})$ system. Worth noting in a report: the lecture's generic scalar-parameter case ($\dot{x} = \theta x + u$) generalizes cleanly here to a vector disturbance because the structure $\ddot{p} = ge_3 - R_z(\psi)u^* + w$ is affine in the unknown w, exactly like the lecture's $g(x,u)\theta$ term.
- **The cascade/two-timescale assumption** ("an inner-loop controller drives (ϕ, θ) to (ϕ_r, θ_r)... time-scale separation... sufficiently large to neglect the coupling") is verbatim the same hierarchical-control justification as `02-control.pdf` (Ch3: "make the inner loop 5-10x faster... so they can be tuned quasi-independently").
- **The LOS guidance law in 3.7** is the same family of "lookahead-distance" guidance mentioned but not derived in `04-guidance-nonlinear.pdf` (Ch8, "Lookahead-distance alternatives: Line-of-sight (LOS) guidance and the L1 guidance law") - Lab3 provides the missing worked formula and turns it into a hands-on exercise.
- **Section 2.8's yaw-tracking-with-zero-steady-state-error** exercise is a miniature version of the integral-action argument in Ch3 (`02-control.pdf`:

PD alone leaves a steady-state offset under a constant disturbance/reference; integral action removes it) - here applied to a general first-order (not pure-integrator) yaw model, so students must also handle the case of a stable non-integrator pole, not just the double-integrator case worked in the lecture.

- **The `wifiControl/*.m` warning about inaccurate position estimates from integrated, optical-flow-derived velocity** is exactly the AR.Drone-specific sensing limitation documented in `03-sensors-estimation.pdf` (no GPS; velocity from vision/optical flow, height from sonar) - a concrete reason why the trajectory-tracking controller designed in Sec. 2 (which assumes clean p, $\dot{p}$ feedback) may perform worse in the real Wi-Fi-control experiment than in simulation, and worth flagging explicitly in a lab report's discussion of results.

6 Open questions / unclear points

- The handout never gives numeric values for the design parameters students must choose (Q, ρ for the LQR-based K, k_w for the adaptation law, the vertical proportional gain k_w in $w_r = -k_w(z - z_d)$ - note this reuses the symbol k_w for two different gains across Sec. 2.5 and Sec. 3.1, a naming collision worth flagging in a report), nor the LOS parameters Δ, V - these are left entirely to student design/tuning, consistent with the lecture's Bryson's-rule "trial and error" tuning philosophy (Ch4) but meaning no "reference" numeric answer exists in the source material.
- The exact content of the `desired_trajectory` and `tt_controller` MATLAB Function blocks inside `ARDroneTTSim.slx/ARDroneTT.slx` could not be inspected (binary Simulink format) - only the handout's prose description of their required I/O and behavior is available.
- The handout states the model's real inputs are $(\phi_r, \theta_r, w_r, \dot{\psi}_r)$ but doesn't show the exact Simulink signal names/units used inside the block (e.g. whether angles are in radians or the AR.Drone's native normalized command range) - would need to open the `.slx` in Simulink to confirm.
- `lib/setupARModel.m` loads six `.mat` files (`ssRoll`, `ssRoll2V`, `ssPitch`, `ssPitch2U`, `ssYaw`, `ssH`) containing the actual identified inner-loop transfer functions/state-space matrices; their numeric content wasn't inspected (binary `.mat`, outside this fork's scope) - if a report needs the literal inner-loop plant model, these files (present in `lib/` of all three DevKit folders) would need to be loaded in MATLAB.